

UNIVERSITÄT SIEGEN

MECHATRONICS

EMBEDDED CONTROL

Lab 3 - Road Following in CoppeliaSim: An Introduction to Robotic Navigation

WS 2023/24

Group 6

1776636 Omar ALMOULD

1707012 Arian SAMADI

1778515 Victor TAUSU

28/01/24



Contents

1	CoppeliaSim	2
1.1	Realistic Physics Simulation:	2
1.2	Comprehensive Sensor Support:	2
1.3	Scripting Versatility:	2
1.4	Customizable Scenarios:	2
1.5	Remote API Capabilities:	2
2	Blob in Image Processing	2
2.1	Convert the Image to Grayscale:	3
2.2	Binarization of the Image:	3
2.3	Finding the Center of the Image using Moments:	3
3	File Structure and Names	3
3.1	constants.py	3
3.2	utils.py	4
3.3	Robot.py	7
3.4	main_controller.py	8
4	Conclusion	10
	References	11

Introduction

Road following, a crucial element in autonomous vehicle navigation, encompasses the design and execution of control algorithms that empower a robot to identify and adhere to a predefined path. This intricate task necessitates a deep comprehension of fundamental robotic principles, including the integration and calibration of various sensors. These sensors might include lidar, cameras, and GPS systems, each contributing vital data for path detection and navigation.

These algorithms empower a robot or an autonomous vehicle to detect, interpret, and follow a predefined path. This task is not trivial; it necessitates a deep understanding of several key areas:

Basic Robotic Principles

This includes knowledge of how robots perceive their environment, make decisions, and execute actions. It involves understanding the mechanics and dynamics of robotic movement, which are crucial for precise navigation and path following.

Sensor Integration

Autonomous vehicles rely heavily on various sensors like cameras, LiDAR (Light Detection and Ranging), GPS, and IMU (Inertial Measurement Units) for environmental perception. Effective road following requires the use of data from these sensors to accurately perceive and interact with the environment. This data is essential for identifying the road, obstacles, and other critical elements.

CoppeliaSim is a robust and flexible robotics simulation platform useful for this task. Its strength lies in the accurate and detailed modelling of robotic sensors and movements, which closely mimic real-world dynamics. This high-fidelity simulation enables developers to rigorously test and refine road following algorithms in a controlled environment before deploying them in actual robots.

The platform's versatility extends to various types of autonomous vehicles, from small-scale robots to full-sized cars, providing a comprehensive test bed for algorithm development.

In tandem with CoppeliaSim, Python programming plays a pivotal role. Python, known for its simplicity and extensive library ecosystem, is utilised to develop the control algorithms and interface with the simulation environment. Libraries such as NumPy for numerical computations, OpenCV for image processing, and robotics-specific packages enhance the development process.

Python's ability to interface with CoppeliaSim allows for a seamless transition from algorithm design to simulation testing.

This approach combines CoppeliaSim's realistic simulation capabilities with Python's programming flexibility, and is essential for advancing road following technologies. It facilitates an understanding of the challenges involved in autonomous navigation and contributes to the development of more efficient, reliable, and safe autonomous vehicles. Through multiple simulation model iterations, developers can use CoppeliaSim to optimise sensor configurations, improve algorithmic efficiency, and ensure robust performance under varying conditions, ultimately leading to advancements in autonomous vehicle technology.

1 Coppeliasim

Coppeliasim stands out in the field of robotics simulation. Designed to be both flexible and powerful, this software offers a 3D simulated environment where robotic models and algorithms can undergo extensive testing and refinement. Its capability to replicate real-world physics and a variety of sensor types accurately makes it highly valued for educational and research activities in robotics.

Key attributes of Coppeliasim:

1.1 Realistic Physics Simulation:

Coppeliasim incorporates a sophisticated physics engine that emulates real-world behaviors and interactions. This aspect is vital for evaluating how robots physically interact with their surroundings.

1.2 Comprehensive Sensor Support:

The software includes a broad spectrum of simulated sensors, like vision sensors, proximity sensors, and inertial measurement units. These are crucial for navigation-related tasks such as road following.

1.3 Scripting Versatility:

The platform enables scripting in multiple languages, including Lua and Python. This versatility allows for a range of programming methods to suit different complexities.

1.4 Customizable Scenarios:

Users have the freedom to create and adjust environments, providing the flexibility to design scenarios that meet specific needs.

1.5 Remote API Capabilities:

Coppeliasim features functionality for controlling simulation elements through external programs, such as Python scripts. This allows for the implementation of more advanced control algorithms and the processing of real-time data.

These comprehensive features make Coppeliasim an exemplary tool for the development and implementation of robotic navigation and control systems. In the context of this lab's focus on road following, it offers an ideal platform for exploring and honing these systems in a controlled, yet realistic, setting.

2 Blob in Image Processing

The concept of a blob in image processing refers to a region in an image which is different in properties, like brightness or color, compared to surrounding regions. Identifying and analyzing blobs can be important in various applications such as object recognition, shape analysis, or tracking movements.

In the context of finding the centroid of a binary blob using OpenCV, the process generally involves several key steps:

2.1 Convert the Image to Grayscale:

This is typically the first step in processing and analyzing images. It simplifies the image by reducing it to a range of gray shades, rather than dealing with full color.

2.2 Binarization of the Image:

Binarization is the process of converting a grayscale image to a binary image. In a binary image, each pixel is marked as either black or white. This is often done using a thresholding technique where pixels with values above a certain threshold are set to white (often representing the blob) and those below to black (representing the background).

2.3 Finding the Center of the Image using Moments:

After binarization, the next step is to calculate the moments of the image. Moments are a set of scalar values derived from an image. They are used to calculate the centroid, the average or mean position of all the points in a shape.

Image Moments:

Image moments help in calculating some features like the center of mass of the object, area of the object, etc. In simple terms, moments are calculated as the weighted average of the pixel intensities of the object.

Centroid Calculation:

The centroid is a point whose coordinates are the average (mean) positions of all the points in a shape. In the context of a binary image, it's the average of all the white pixel's coordinates.

The formulae for calculating the centroid c_x and c_y (the x and y coordinates of the centroid) are given by:

$$c_x = \frac{M_{10}}{M_{00}}, \quad c_y = \frac{M_{01}}{M_{00}}$$

where M_{10} , M_{01} , and M_{00} are moments of the image. M_{00} is actually the area (or the sum of all white pixels in the binary case), M_{10} is the sum of the x coordinates of all white pixels, and M_{01} is the sum of the y coordinates.

3 File Structure and Names

3.1 constants.py

The constants.py file is a configuration file that includes various parameters for robot control in the simulation environment. Here's a breakdown of each constant defined in the file and its purpose:

- **PID Controller Gains (KP, KI, KD):** These are the gains for a PID (Proportional-Integral-Derivative) controller.
 - **KP (Proportional gain):** Influences the control action proportional to the error. A higher KP results in a larger change in the control output for a given change in the error.
 - **KI (Integral gain):** Addresses the accumulated past errors. It integrates the error over time, thus helping to eliminate steady-state errors.
 - **KD (Derivative gain):** Predicts future error based on the current rate of error change. It helps in dampening or preventing overshoot by considering the error's rate of change.
- **Resolution (RES_X, RES_Y):** These constants define the resolution of the vision sensor in the simulation.
 - **RES_X and RES_Y:** represent the horizontal and vertical resolution, respectively. The resolution determines how many pixels are used to represent the vision sensor's field of view.
- **Threshold Limits (THRESH_UL, THRESH_LL):** These are likely used for image processing, specifically for setting upper (THRESH_UL) and lower (THRESH_LL) threshold limits.
- **Default Velocity (DEF_VEL):** This constant sets a default velocity for the robots. This velocity could be the baseline speed at which the robots move in the simulation unless otherwise controlled by the algorithm.
- **Robots ID Strings List (R):** This is a list of identifiers for the robots in the CoppeliaSim simulation. Each string in the list ("/Pioneer3DX[0]", "/Pioneer3DX[1]", etc.) corresponds to a specific robot model within the simulation. These IDs are used to reference and control individual robots programmatically.

Overall, constants.py serves as a centralized location for parameters that are likely used across various parts of the road following simulation, such as in the robot control algorithms. Having these values in a single file makes it easier to adjust and maintain the parameters without searching through the entire codebase.

3.2 utils.py

The utils.py file contains several utility functions that are essential for image processing and analysis in the context of robotic control. Let's examine each function in detail:

- **getRawImage(sim, visionSensor, resX, resY):**
 - **Purpose:** To retrieve the raw image data from a vision sensor within the simulation environment.
 - **Parameters:**
 - * **sim:** The simulation handle, which is used to interact with the CoppeliaSim environment.
 - * **visionSensor:** The specific vision sensor from which the image is to be captured.

- * **resX, resY:** Desired resolution for the image capture. If set to 0, the default resolution of the vision sensor is used.
- **Functionality:** It uses the `sim.getVisionSensorImg` function to obtain the image data from the specified vision sensor at the given resolution.
- **Returns:** The raw image data and its resolution.
- **getThresh(image, resX, resY):**
 - **Purpose:** To process the raw image data and apply thresholding to create a binary image.
 - **Parameters:**
 - * **image:** The raw image data.
 - * **resX, resY:** The resolution of the image.
 - **Functionality:**
 - * The function first converts the raw image data into a NumPy array of uint8 type and reshapes it according to the specified resolution.
 - * The image is then transformed to grayscale and thresholded using OpenCV functions. Thresholding is based on predefined limits (`THRESH_LL` and `THRESH_UL` from `constants.py`) to create a binary image.
 - **Returns:** The thresholded binary image and the original RGB image.
- **getCentroid(thresh, desCentroid):**
 - **Purpose:** To find the centroid of a binary image, which is a key step in determining the robot's position relative to the path.
 - **Parameters:**
 - * **thresh:** The thresholded binary image.
 - * **desCentroid:** The desired centroid position, typically the center of the image.
 - **Functionality:**
 - * The function calculates the moments of the binary image using OpenCV's `cv2.moments` function.
 - * It then computes the centroid (center of mass) of the white areas in the binary image. If the image is empty (no white areas), the desired centroid is used as a fallback.
 - **Returns:** The actual X and Y coordinates of the centroid.
- **showImage(viewStr, img):**
 - **Purpose:** To display an image in a window, useful for debugging or visualization purposes.
 - **Parameters:**
 - * **viewStr:** The name of the window in which the image will be displayed.

- * **img:** The image to be displayed.
- **Functionality:**
 - * Displays the image using OpenCV's `imshow` and `waitKey` functions. This allows real-time visualization of what the robot's vision sensor is seeing.

In summary, `utils.py` provides essential image processing tools for the robotic control system, particularly for tasks like vision-based navigation or path following. These functions facilitate the extraction and analysis of visual data from the robot's sensors, which is crucial for implementing algorithms like road following in autonomous robotics.

The `Robot` class in `robot.py` is crucial for simulating a robot's behavior in a controlled environment. It utilizes image processing (from `utils.py`) and PID control to follow a path. Here's a breakdown of each function in the class:

A. `init(self, name, simHandle):`

- **Purpose:** Constructor for initialising an instance of the `Robot` class.
- **Functionality:** Initializes instance variables, including the robot's name (`__name`), simulation handle (`__simHandle`), motor handles (`__leftMotor` and `__rightMotor`), and vision sensor handle (`__visionSensor`). Initializes PID controller parameters (`__error_prev` and `__integral`) for the robot.

B. `name` property (with `@property`):

- **Purpose:** Provides read-only access to the robot's name.
- **Functionality:** Returns the private `__name` variable of the robot instance.

C. `getDiffFactor(self, desCentroid, actCentroid):`

- **Purpose:** Calculates the differential factor for the PID controller based on the deviation of the actual centroid from the desired centroid.
- **Functionality:** Computes the error as the difference between the desired centroid (`desCentroid`) and the actual centroid (`actCentroid`). Updates the integral and derivative components of the PID controller. Returns the differential factor, which is a scaled version of the PID output.

D. `move(self, velocity, diffFactorX):`

- **Purpose:** Controls the robot's motors based on the given velocity and differential factor.
- **Functionality:** Calculates the velocities for the left and right motors using the base velocity (`velocity`) and the differential factor (`diffFactorX`). Sets the calculated velocities to the robot's motors using the simulation handle.

E. `update(self, velocity=DEF_VEL, showImg=False):`

- **Purpose:** Main function for updating the robot's state, including processing vision sensor data and moving the robot.
- **Functionality:** Captures an image from the vision sensor and processes it to determine the actual centroid. Calculates the differential factor for PID control and updates the robot's motor velocities accordingly. Optionally displays the processed image if `showImg` is `True`.

F. `stop(self)`:

- **Purpose:** Stops the robot by setting its motor velocities to zero.
- **Functionality:** Calls the simulation handle to set the target velocities of both motors to zero, stopping the robot.

3.3 Robot.py

The `robot.py` file defines a `Robot` class that holds the functionalities and behaviors of a robot within a simulated environment. Let's break down each function in the class for a better understanding:

`__init__(self, name, simHandle)`:

Purpose: Constructor for initializing an instance of the `Robot` class.

Functionality:

- Initializes instance variables including the robot's name (`__name`), simulation handle (`__simHandle`), motor handles (`__leftMotor` and `__rightMotor`), and vision sensor handle (`__visionSensor`).
- Initializes PID controller parameters (`__error_prev` and `__integral`) for the robot.

`name` property (decorated with `@property`):

Purpose: Provides a read-only access to the robot's name.

Functionality: Returns the private `__name` variable of the robot instance.

`getDiffFactor(self, desCentroid, actCentroid)`:

Purpose: Calculates the differential factor for the PID controller based on the deviation of the actual centroid from the desired centroid.

Functionality:

- Computes the error as the difference between the desired centroid (`desCentroid`) and the actual centroid (`actCentroid`).
- Updates the integral and derivative components of the PID controller.
- Returns the differential factor, which is a scaled version of the PID output.

```
move(self, velocity, diffFactorX):
```

Purpose: Controls the robot's motors based on the given velocity and differential factor.

Functionality:

- Calculates the velocities for the left and right motors using the base velocity (`velocity`) and the differential factor (`diffFactorX`).
- Sets the calculated velocities to the robot's motors using the simulation handle.

```
update(self, velocity=DEF_VEL, showImg=False):
```

Purpose: Main function for updating the robot's state, including processing vision sensor data and moving the robot.

Functionality:

- Captures an image from the vision sensor and processes it to determine the actual centroid.
- Calculates the differential factor for PID control and updates the robot's motor velocities accordingly.
- Optionally displays the processed image if `showImg` is `True`.

```
stop(self):
```

Purpose: Stops the robot by setting its motor velocities to zero.

Functionality: Calls the simulation handle to set the target velocities of both motors to zero, stopping the robot.

In summary, the `Robot` class in `robot.py` is crucial for simulating a robot's behavior in a controlled environment. It utilizes image processing (from `utils.py`) and PID control to follow a path, with methods to calculate control actions based on vision sensor data, and to update the robot's state in each simulation step. The class provides a structured approach to implementing autonomous navigation tasks like road following in robotics simulations.

3.4 main_controller.py

The `main_controller.py` script is the main executable for controlling the robots in the simulation environment. It integrates various components like robot control, simulation handling, and image processing. Here's a breakdown of the main concepts and functions in this script:

Import Statements

Imports necessary modules and classes: The script imports the `Robot` class from `robot.py`, the `RemoteAPIClient` for interacting with Coppeliasim, and `constants` from `constants.py`. It also imports standard libraries like `numpy`, `cv2`, and `time`.

The main() Function

Initialize the Simulation Connection:

- `client = RemoteAPIClient()`: Creates an instance of the `RemoteAPIClient` to interact with CoppeliaSim.
- `sim = client.getObject('sim')`: Retrieves the simulation object for controlling the simulation.

Robot Instantiation:

- Robots (`r0`, `r1`, `r2`, `r3`, `r4`) are called at different instants using the `Robot` class, with each robot being identified by a unique ID from the list `R` in `constants.py`.

Simulation Control:

- `sim.setStepping(True)`: Configures the simulation for stepped progression.
- `sim.startSimulation()`: Starts the simulation.

Simulation Loop:

- A loop runs for a fixed simulation time, where each robot's state is updated in each step.
- `rX.update(velocity=DEF_VEL)`: Calls the `update` method for each robot, which processes the vision sensor data and moves the robot. The `velocity` parameter controls the robot's speed, and `showing` determines whether to display the processed image.

Step Through the Simulation:

- `sim.step()`: Advances the simulation by one step.

Cleanup:

- `cv2.destroyAllWindows()`: Closes any OpenCV windows opened for image display.
- `sim.stopSimulation()`: Stops the simulation.

Main Execution Check

- `if __name__ == '__main__': main()`: This is a common Python statement for running the main function when the script is executed directly.

Concepts Used in the Script

- Modular Design: The script is structured in a modular way, separating the initialization, main loop, and cleanup stages.
- Robot Control: Uses the `Robot` class to store the control logic for each robot, demonstrating object-oriented programming principles.

- **Simulation Handling:** Interacts with the CoppeliaSim environment via the `RemoteAPIClient`, demonstrating how external simulation environments can be controlled programmatically.
- **Real-time Updating:** The loop in the `main()` function represents a real-time control loop where the state of each robot is updated continuously based on sensor data.

In summary, `main_controller.py` serves as the central script for running the road following simulation. It initializes the simulation environment, controls the robots based on sensor data, and manages the simulation's progression, demonstrating key concepts in robotics control and simulation handling.

4 Conclusion

In conclusion, the developed robot simulation system for road following stands as a comprehensive and meticulously crafted framework. The `main_controller.py` script orchestrates the simulation, demonstrating a modular design that separates initialization, main loop execution, and cleanup stages.

Each component, including `constants.py`, `utils.py`, `robot.py`, and the integration with CoppeliaSim, plays a distinct role. The Python programming language, with its libraries such as NumPy and OpenCV, synergizes seamlessly with CoppeliaSim, providing a robust foundation for testing and refining road following algorithms.

The importance of the Robot class in `robot.py` cannot be overstated, as it holds critical functionalities for simulating a robot's behavior. By leveraging image processing techniques and PID control, the class contributes significantly to autonomous navigation tasks.

The utilization of CoppeliaSim, with its realistic physics simulation, sensor support, and scripting versatility, elevates the platform's standing in the field of robotics simulation. The integrated approach of Python and CoppeliaSim, combined with a focus on road following, ensures a controlled yet realistic environment for algorithm development.

Additionally, the exploration of image processing concepts, specifically blobs and centroids, sheds light on essential techniques for object recognition. The use of OpenCV and Python adds a layer of sophistication to the system, facilitating tasks such as shape analysis and tracking movements.

In essence, this research contributes to the advancement of autonomous vehicle technology. The rigorous testing and optimization of sensor configurations and algorithmic efficiency in a simulated environment pave the way for the development of more efficient, reliable, and safe autonomous vehicles.

References

- [1] Correll, N., Hayes, B., et al. (2019). *Introduction to Autonomous Robots: Mechanisms, Sensors, Actuators, and Algorithms*.
- [2] Siciliano, B., Sciavicco, L. (2010). *Robotics: Modelling, Planning and Control*.
- [3] Dudek, G., Jenkin, M. (2000). *Computational Principles of Mobile Robotics*.